

1. Introduction

we looked at the time complexity of the operations that were built in Milestone 1 of the Campus Resource Reservation System. The Campus Resource Reservation System uses a linked list to hold reservations a queue to hold waiting requests and a stack to hold cancellation history.

For this analysis:

n = number of active reservations

r = number of resources

2. Reservation Insertion

Time Complexity: $O(n)$

I see that active reservations are kept in a linked list. When a new reservation is added the system scans the list from the beginning all the way to the end before placing the reservation. If there are n reservations the system may have to check all n nodes. Because of this, the worst-case time it takes to add a reservation is $O(n)$.

3. Reservation Removal

Time Complexity: $O(n)$

When a reservation is cancelled the program looks through the linked list using the reservation ID. In the worst case, it might check all n active reservations before finding the one that was asked for or, before realizing it isn't there. Removing the node takes $O(1)$ time once the node is found but the search part makes the whole process $O(n)$.

4. Waiting-List Processing

Time Complexity: $O(n + r)$

The waiting list behaves like a queue. This means that retrieving an item from the waiting list and removing it takes a fixed amount of time which we call $O(1)$. When a new request arrives, the system also creates a reservation. In the implementation the system looks through the reservation list and the resource vector. Checking these can takes $O(n)$ for the reservation list and $O(r)$ for the resource vector. Therefore, the maximum time needed to process a waiting request is $O(n + r)$.

5. Undo Cancellation

Time Complexity: $O(n)$

Cancelled reservations are kept in a stack. Taking the most recent cancelled reservation from the stack takes constant time, $O(1)$. After that the current code walks through the reservation linked list to add the restored reservation back in. That walk can take time $O(n)$ so the entire undo operation can be as slow, as $O(n)$.

6. Complexity Summary

Operation	Data Structure	Time Complexity
Reservation insertion	Linked list	$O(n)$
Reservation removal	Linked list	$O(n)$
Waiting-list processing	Queue + linked list + resource vector	$O(n + r)$
Undo cancellation	Stack + linked list	$O(n)$

7. Conclusion

Milestone 1 implementation uses linked list, queue and stack for reservation operations. Operations have linear worst-case complexity. The operations have linear worst-case complexity when the implementation needs to search or traverse existing data.

.